

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN
Thực hiện phần cứng cho giải thuật xử lý ảnh bằng FPGA

GVHD: Hồ Ngọc Diễm

Lớp: CE213.Q23

Họ và tên	MSSV
Trần Đức Cường	23520208
Lương Ngọc Đại	23520215

Mục Lục

CHƯƠNG 1:GIỚI THIỆU.....	3
1.1 Đặt vấn đề	3
1.2 Mục tiêu đề tài.....	3
1.3 Nội dung thực hiện	3
1.4 Phạm vi đề tài	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	4
2.1 Tổng quan xử lí ảnh số.....	4
2.2 Chuyển ảnh màu sang ảnh xám (gray scale):	5
2.3 Xử lí ảnh bị nhiễu (median filter)	5
2.4 Cân bằng độ tương phản (thuật toán Histogram Equalization):	7
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG	9
3.1 Sơ đồ khối hệ thống	9
3.2 Thiết kế từng module	10
3.2.1 Module RGB to Gray	10
3.2.2 Module Median Filter.....	11
3.2.3 Module Histogram Equalization	13
3.3 Bộ nhớ và lưu trữ ảnh	19
3.4 Lưu đồ thuật toán.....	21
CHƯƠNG 4. KẾT QUẢ MÔ PHỎNG VÀ THỰC NGHIỆM	23
4.1 Mô phỏng	23
Mô phỏng thuật toán gray scale:.....	23
Mô phỏng thuật toán median filter:	24
Mô phỏng thuật toán histogram 1:.....	25
Mô phỏng thuật toán histogram 2:.....	26
Mô phỏng thuật toán histogram và median filter:.....	27
4.2 Thực nghiệm.....	28
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	28
TÀI LIỆU THAM KHẢO	30

CHƯƠNG 1:GIỚI THIỆU

1.1 Đặt vấn đề

Trong những năm gần đây, xử lý ảnh số đã trở thành một lĩnh vực quan trọng và được ứng dụng rộng rãi trong nhiều hệ thống hiện đại như camera giám sát, xe tự hành, y tế, robot và trí tuệ nhân tạo. Các hệ thống này yêu cầu khả năng xử lý dữ liệu hình ảnh nhanh, chính xác và hoạt động theo thời gian thực.

Một số ứng dụng tiêu biểu của xử lý ảnh gồm:

- Nhận diện khuôn mặt trong camera an ninh.
- Hỗ trợ quan sát và chẩn đoán hình ảnh trong y tế.
- Nhận diện vật cản trong xe tự hành.
- Tiền xử lý dữ liệu cho các hệ thống AI Vision.

Trong các nền tảng phần cứng hiện nay, FPGA có ưu điểm về khả năng xử lý song song, tốc độ cao và khả năng hoạt động thời gian thực, rất phù hợp cho các bài toán xử lý ảnh. So với xử lý bằng phần mềm trên CPU, FPGA giúp tăng tốc đáng kể các phép toán ảnh và giảm độ trễ hệ thống.

Xuất phát từ những yêu cầu trên, đề tài “Thiết kế và thực hiện các thuật toán xử lý ảnh trên FPGA” được thực hiện nhằm nghiên cứu và triển khai các thuật toán xử lý ảnh cơ bản trên nền tảng FPGA DE2. Các thuật toán được lựa chọn gồm chuyển đổi ảnh xám (Grayscale), lọc trung vị (Median Filter) và cân bằng histogram (Histogram Equalization) nhằm cải thiện chất lượng ảnh và làm nền tảng cho các hệ thống xử lý ảnh phức tạp hơn.

1.2 Mục tiêu đề tài

Đề tài hướng đến các mục tiêu chính sau:

- Nghiên cứu nguyên lý hoạt động của các thuật toán xử lý ảnh cơ bản.
- Thực hiện chuyển đổi ảnh màu RGB sang ảnh grayscale 8-bit.
- Thực hiện khử nhiễu ảnh bằng thuật toán Median Filter.
- Thực hiện tăng cường độ tương phản ảnh bằng Histogram Equalization.
- Thiết kế và mô phỏng các thuật toán bằng ngôn ngữ Verilog HDL.
- Triển khai và kiểm thử hệ thống trên FPGA DE2.
- Đánh giá kết quả xử lý ảnh sau khi thực hiện các thuật toán.

1.3 Nội dung thực hiện

Các nội dung chính được thực hiện trong đề tài gồm:

- Tìm hiểu cơ sở lý thuyết về xử lý ảnh số và FPGA.
- Nghiên cứu nguyên lý của các thuật toán: Grayscale, Median Filter, Histogram Equalization
- Thiết kế phần cứng bằng Verilog HDL.
- Xây dựng các module xử lý ảnh trên FPGA.
- Mô phỏng và kiểm tra hoạt động của hệ thống bằng ModelSim.
- Nạp thiết kế lên kit FPGA DE2 để kiểm thử thực tế.
- Truyền dữ liệu ảnh giữa máy tính và FPGA.
- Đánh giá chất lượng ảnh sau xử lý.

1.4 Phạm vi đề tài

Trong phạm vi đề tài, hệ thống được giới hạn như sau

- Xử lý ảnh tĩnh, không xử lý video thời gian thực.
- Ảnh đầu vào là ảnh grayscale 8-bit hoặc ảnh RGB được chuyển đổi sang grayscale.
- Kích thước ảnh được lưu và xử lý trong bộ nhớ SDRAM.
- Sử dụng kit FPGA DE2 của hãng Altera (Intel FPGA).
- Không thực hiện các thuật toán AI hoặc nhận dạng đối tượng phức tạp.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan xử lý ảnh số

Xử lý ảnh số (Digital Image Processing) là lĩnh vực nghiên cứu các phương pháp sử dụng máy tính hoặc phần cứng số để xử lý, phân tích và cải thiện chất lượng ảnh.

Ảnh sau khi được thu nhận từ camera hoặc cảm biến sẽ được biểu diễn dưới dạng dữ liệu số. Các thuật toán xử lý ảnh sẽ tác động lên dữ liệu này nhằm:

- Cải thiện chất lượng ảnh.
- Loại bỏ nhiễu.
- Tăng độ tương phản.
- Trích xuất thông tin.
- Hỗ trợ nhận dạng và phân tích ảnh.

Ngày nay, xử lý ảnh số được ứng dụng rộng rãi trong nhiều lĩnh vực như:

- Camera giám sát.
- Y tế.
- Xe tự hành.
- Nhận dạng khuôn mặt.

- Công nghiệp tự động hóa.
- Trí tuệ nhân tạo (AI).

Ảnh số là ảnh được biểu diễn dưới dạng ma trận các điểm ảnh (pixel). Mỗi pixel chứa thông tin về cường độ sáng hoặc màu sắc tại vị trí tương ứng trong ảnh. Ảnh có càng nhiều pixel thì ảnh càng sắc nét.

2.2 Chuyển ảnh màu sang ảnh xám (gray scale):

Ảnh màu RGB bao gồm ba thành phần màu Red (R); Green (G); Blue (B). Mỗi kênh màu được biểu diễn bằng 8-bit. Do đó, một pixel ảnh màu sẽ có tổng cộng 24-bit dữ liệu.

Để chuyển đổi ảnh RGB sang ảnh grayscale ta chuyển ảnh màu RGB sang ảnh bitmap

Tách từng pixel ảnh màu RGB thành 3 kênh màu R, G, B. Mỗi kênh màu là 8 bit. Sau đó ghép 3 kênh thành 24bit của bitmap. Như vậy mỗi pixel ảnh màu sẽ được chuyển thành 24bit với định dạng 3 màu R,G,B tương ứng R [23:16] G[15:8] B[7:0]

Xử dụng công thức chuẩn luminance để điều chỉnh giá trị từng kênh R,G,B để thành ảnh xám (gray scale)

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

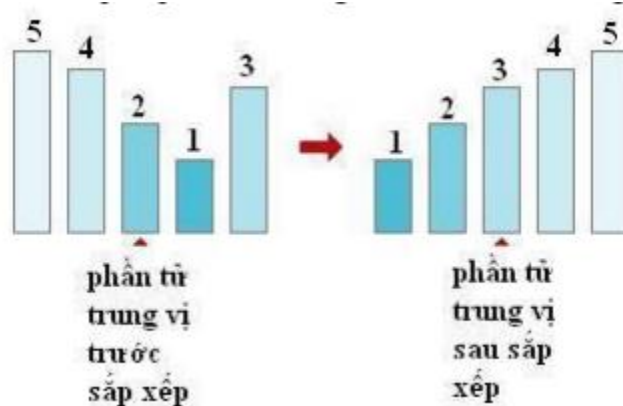
Có tham số độ sáng để điều chỉnh. Các giá trị sẽ được cộng thêm tham số độ sáng.

2.3 Xử lý ảnh bị nhiễu (median filter)

Dùng bộ lọc trung vị (median filter) để xử lý

Lọc trung vị là một kỹ thuật lọc phi tuyến (non-linear), nó khá hiệu quả đối với hai loại nhiễu: nhiễu đốm (speckle noise) và nhiễu muối tiêu (salt-pepper noise). Kỹ thuật này là một bước rất phổ biến trong xử lý ảnh.

Ý tưởng chính của thuật toán lọc trung vị như sau. Đầu tiên ta sử dụng một cửa sổ lọc (ma trận 3×3) quét qua lần lượt từng điểm ảnh của ảnh đầu vào input. Tại vị trí mỗi điểm ảnh lấy giá trị của các điểm ảnh tương ứng trong vùng 3×3 của ảnh gốc “lấp” vào ma trận lọc. Sau đó sắp xếp các điểm ảnh trong cửa sổ này theo thứ tự (tăng dần hoặc giảm dần tùy ý). Cuối cùng, gán điểm ảnh nằm chính giữa (Trung vị) của dãy giá trị điểm ảnh đã được sắp xếp ở trên cho giá trị điểm ảnh đang xét của ảnh đầu ra output.



Ví dụ về lọc trung vị trong xử lý ảnh:

4	4	3	2	1	1
4	4	3	2	1	1
3	3	1	2	4	4
5	5	1	6	2	2
2	2	3	5	6	6
2	2	3	5	6	6

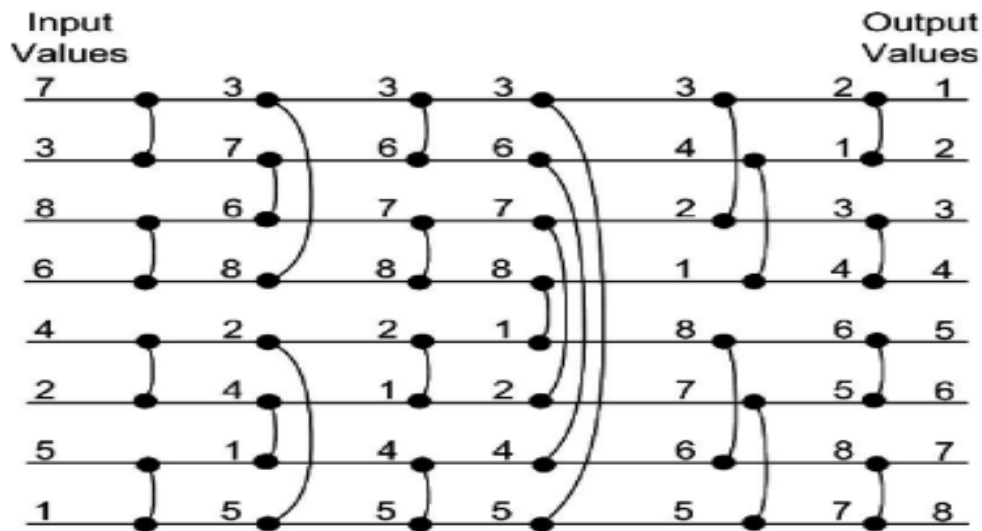
4 4 3 4 4 3 3 3 1

Arrange it in ascending or descending order

4 4 4 4 3 3 3 3 1

Để tăng tốc độ xử lý trên FPGA, đề tài sử dụng Sort Network để sắp xếp dữ liệu cho bộ lọc trung vị.

Sort network (mạng phân loại/mạng sắp xếp) là một mô hình tính toán song song, bao gồm các bộ so sánh (comparators) kết nối cố định với nhau để sắp xếp một số lượng đầu vào xác định. Khác với thuật toán sắp xếp thông thường, thứ tự so sánh của sort network được định sẵn, không phụ thuộc vào kết quả các bước trước đó, giúp tối ưu cho phần cứng và xử lý song song.



2.4 Cân bằng độ tương phản (thuật toán Histogram Equalization):

Thuật toán histogram equalization là một phương pháp xử lý ảnh kỹ thuật số giúp tăng cường độ tương phản và mở rộng dải phân bố cường độ sáng. Kỹ thuật này trải đều các mức xám, làm cho hình ảnh rõ nét và chi tiết hơn, đặc biệt khi ảnh bị quá tối hoặc quá sáng

Nguyên lý hoạt động: Thuật toán Histogram Equalization gồm các bước chính:

- Xây dựng histogram gốc:
Histogram là biểu đồ biểu diễn sự phân bố mức xám trong ảnh. Đối với ảnh grayscale 8-bit thì sự phân bố mức xám r trong khoảng $0 \leq r \leq 255$.
Để xây dựng Histogram gốc ta tiến hành thống kê tần suất xuất hiện $h(r_k)$ của từng mức xám r_k trong ảnh. Sau đó ta tính xác suất phân phối:

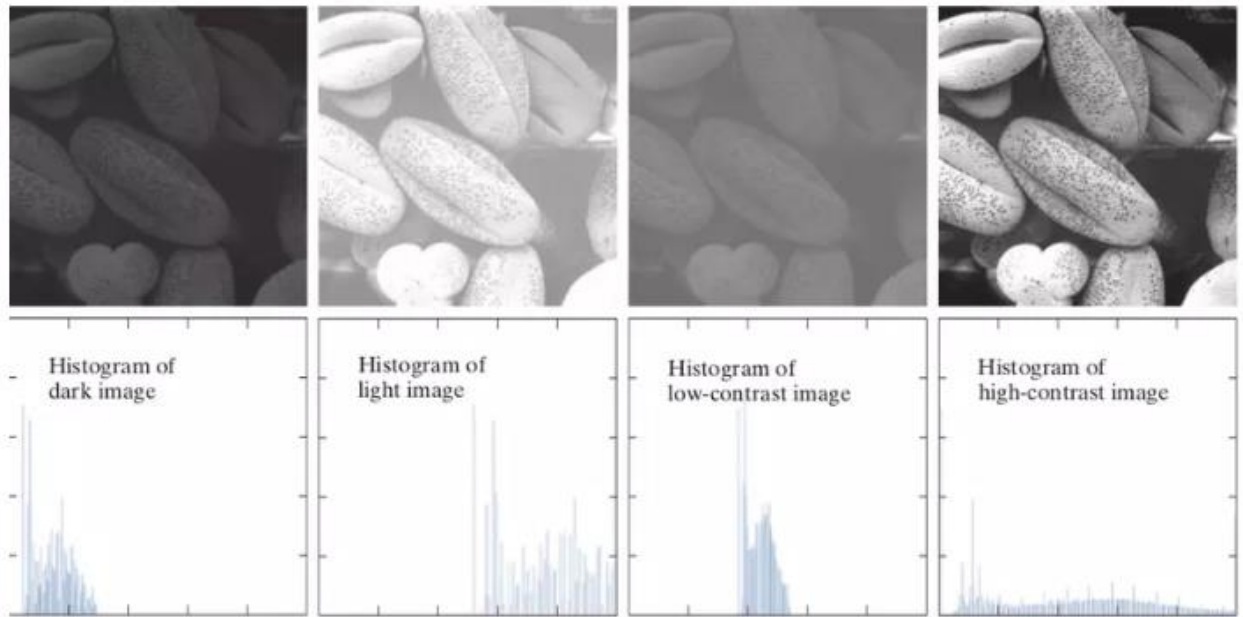
$$p(r_k) = \frac{h(r_k)}{N}$$

Trong đó: $p(r_k)$ là xác suất xuất hiện mức xám r_k trong ảnh.

Độ tương phản là sự khác biệt giữa vùng sáng và vùng tối trong ảnh.

Histogram tập trung hẹp $\rightarrow$ độ tương phản thấp.

Histogram trải rộng $\rightarrow$ độ tương phản cao.



(ví dụ về histogram với độ tương phản khác nhau)

Histogram Equalization giúp trải rộng histogram nhằm tăng độ tương phản tổng thể của ảnh.

- Tính hàm phân phối tích lũy:

Sau khi tính được xác suất phân bố, tiến hành tính hàm phân phối tích lũy (CDF – Cumulative Distribution Function)

$$CDF(r_k) = \sum_{j=0}^k p(r_j)$$

Trong đó: $CDF(r_k)$ là tổng xác suất từ mức xám 0 đến mức xám r_k , luôn tăng dần từ $0 \rightarrow 1$.

Ý nghĩa: Biểu diễn xác suất tích lũy, dùng để ánh xạ mức xám.

- Tính hàm biến đổi mức xám

Sau khi tính được CDF, mức xám mới sẽ được tính dựa vào công thức:

$$s_k = (L - 1) \times CDF(r_k)$$

Trong đó s_k là mức xám mới, L là số mức xám của ảnh.

Đối với ảnh grayscale 8-bit L sẽ bằng 256.

Do đó

$$s_k = 255 \times CDF(r_k)$$

Kết quả sẽ được làm tròn thành số nguyên để tạo thành giá trị pixel mới.

- Bước cuối là tạo ảnh đầu ra. Ở bước cuối cùng, mỗi pixel có giá trị mức xám ban đầu r_k sẽ được thay thế bằng giá trị mới s_k . Sau quá trình ánh xạ Histogram của

ảnh sẽ được phân bố đều hơn, độ tương phản của ảnh được cải thiện và các chi tiết trong vùng tối và vùng sáng trở nên rõ hơn.

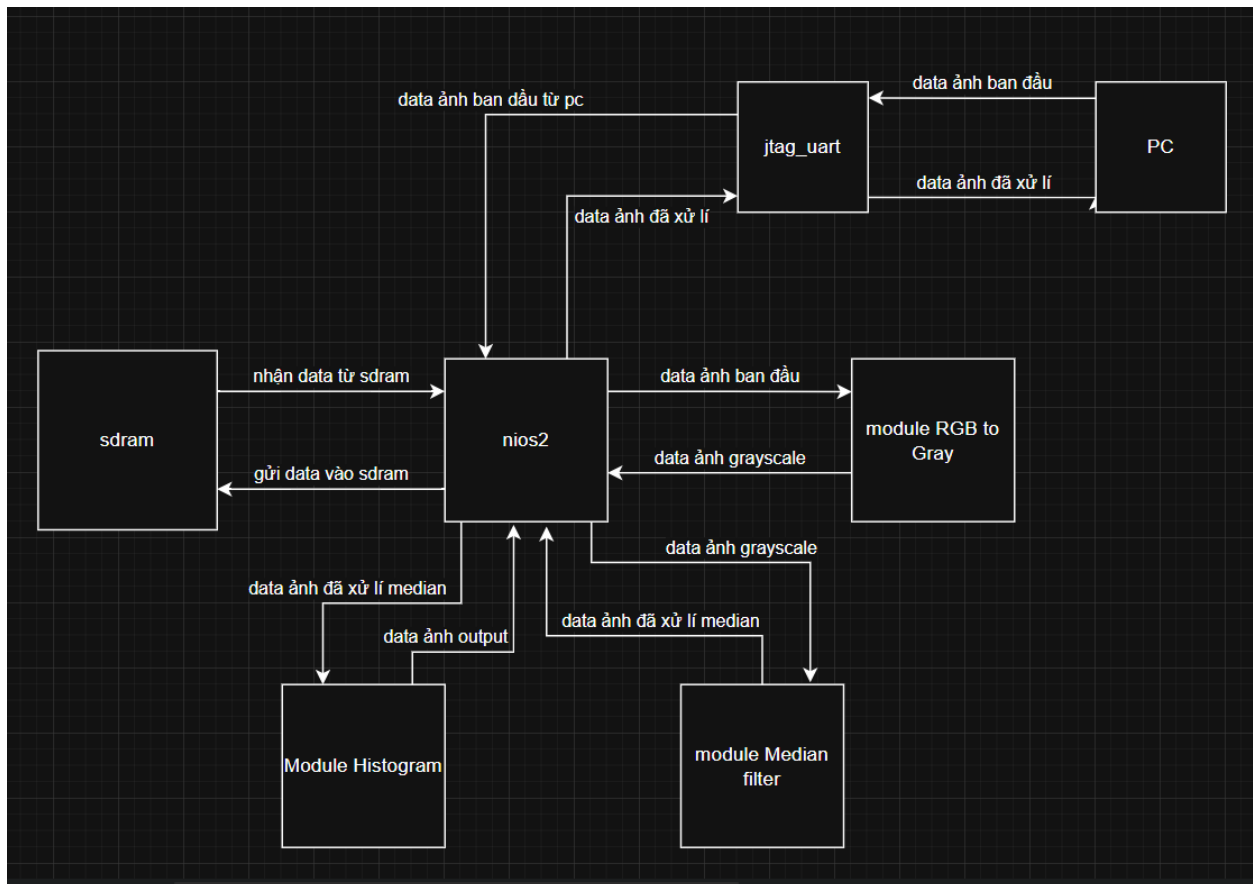
$$r_k = s_k$$

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1 Sơ đồ khối hệ thống

Hệ thống xử lý ảnh trên FPGA được thiết kế theo mô hình xử lý tuần tự gồm các khối chức năng chính: chuyển đổi ảnh RGB sang grayscale, lọc nhiễu bằng Median Filter và tăng cường độ tương phản bằng Histogram Equalization.

Ảnh đầu vào được truyền từ máy tính xuống FPGA và lưu vào bộ nhớ SDRAM. Sau đó dữ liệu ảnh lần lượt đi qua các module xử lý trước khi xuất ra ảnh kết quả.



Sơ đồ khối tổng quát của hệ thống như sau:

Chức năng các khối

- Nios2: CPU điều khiển các luồng dữ liệu và các khối sdram, jtag_uart
- SDRAM: lưu data của ảnh
- Jtag_uart: giao tiếp truyền nhận data ảnh từ PC <-> hệ thống
- RGB to Gray: chuyển đổi ảnh màu RGB sang ảnh grayscale 8-bit.
- Median Filter: khử nhiễu ảnh bằng bộ lọc trung vị.
- Histogram Equalization: cải thiện độ tương phản của ảnh.

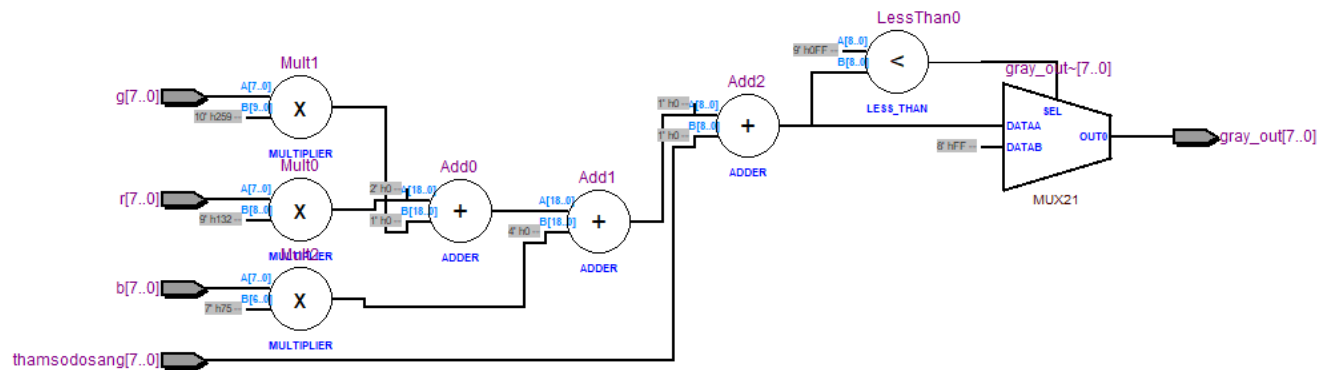
3.2 Thiết kế từng module

3.2.1 Module RGB to Gray

- Verilog

```
module rgb_to_gray(  
input [7:0] r,g,b,  
input [7:0] thamsodosang,  
output [7:0] gray_out  
);  
//  $Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$   
// => Nhân lên với 1024  
wire [17:0] r_after, g_after, b_after;  
assign r_after = r*10'd306;  
assign g_after = g*10'd601;  
assign b_after = b*10'd117;  
wire [19:0] sum = r_after + g_after + b_after;  
wire [7:0] sum_after = sum[17:10]; // dịch phải 10 bit  
//Kiểm tra xem có tràn hay không  
wire [8:0] sum_dosang = sum_after + thamsodosang;  
assign gray_out = (sum_dosang > 9'd255) ? 8'd255 : sum_dosang[7:0]; // Nếu độ sáng  
lớn hơn > 255 thì lấy 255  
endmodule
```

- Netlist viewer



3.2.2 Module Median Filter

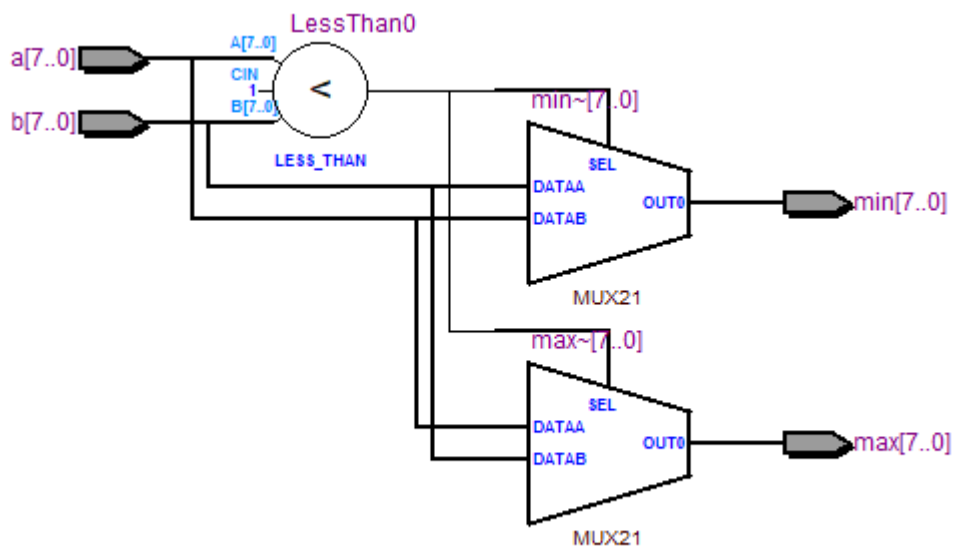
- Verilog swap

```

module swap(
input [7:0] a, input [7:0] b,
output [7:0] min, output [7:0] max
);
assign min = (a <= b) ? a : b;
assign max = (a <= b) ? b : a;
endmodule

```

- Netlist viewer



- Verilog Median

```

module median (

```

```

input [7:0] p0, p1, p2, p3, p4, p5, p6, p7, p8,
output [7:0] out_median
);
// day trung gian cho 9 tang
wire [7:0] s1[0:8], s2[0:8], s3[0:8], s4[0:8], s5[0:8],
s6[0:8], s7[0:8], s8[0:8], s9[0:8];
//stage 1
swap sw1_0(p0, p1, s1[0], s1[1]);
swap sw1_1(p2, p3, s1[2], s1[3]);
swap sw1_2(p4, p5, s1[4], s1[5]);
swap sw1_3(p6, p7, s1[6], s1[7]);
assign s1[8] = p8;
// stage 2
swap sw2_0(s1[1], s1[2], s2[1], s2[2]);
swap sw2_1(s1[3], s1[4], s2[3], s2[4]);
swap sw2_2(s1[5], s1[6], s2[5], s2[6]);
swap sw2_3(s1[7], s1[8], s2[7], s2[8]);
assign s2[0] = s1[0];
// stage 3
swap sw3_0(s2[0], s2[1], s3[0], s3[1]);
swap sw3_1(s2[2], s2[3], s3[2], s3[3]);
swap sw3_2(s2[4], s2[5], s3[4], s3[5]);
swap sw3_3(s2[6], s2[7], s3[6], s3[7]);
assign s3[8] = s2[8];
// stage 4
swap sw4_0(s3[0], s3[2], s4[0], s4[2]);
swap sw4_1(s3[1], s3[3], s4[1], s4[3]);
swap sw4_2(s3[4], s3[6], s4[4], s4[6]);
swap sw4_3(s3[5], s3[7], s4[5], s4[7]);
assign s4[8] = s3[8];
// stage 5
swap sw5_0(s4[1], s4[2], s5[1], s5[2]);
swap sw5_1(s4[5], s4[6], s5[5], s5[6]);
swap sw5_2(s4[0], s4[4], s5[0], s5[4]);
swap sw5_3(s4[3], s4[7], s5[3], s5[7]);
assign s5[8] = s4[8];
// stage 6
swap sw6_0(s5[1], s5[5], s6[1], s6[5]);
swap sw6_1(s5[2], s5[6], s6[2], s6[6]);
swap sw6_2(s5[0], s5[8], s6[0], s6[8]);

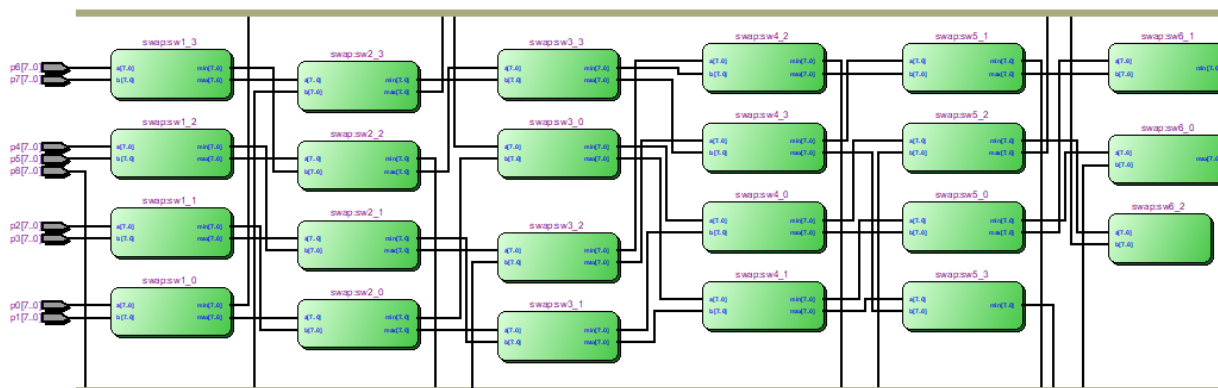
```

```

assign s6[3] = s5[3];
assign s6[4] = s5[4];
assign s6[7] = s5[7];
// stage 7
swap sw7_0(s6[2], s6[4], s7[2], s7[4]);
swap sw7_1(s6[3], s6[5], s7[3], s7[5]);
assign s7[0] = s6[0];
assign s7[1] = s6[1];
assign s7[6] = s6[6];
assign s7[7] = s6[7];
assign s7[8] = s6[8];
// stage 8
swap sw8_0(s7[2], s7[3], s8[2], s8[3]);
swap sw8_1(s7[4], s7[5], s8[4], s8[5]);
assign s8[0] = s7[0];
assign s8[1] = s7[1];
assign s8[6] = s7[6];
assign s8[7] = s7[7];
assign s8[8] = s7[8];
// stage 9
wire [7:0] u9_3, u9_5;
swap sw9_0(s8[3], s8[5], s9[3], s9[5]);
swap sw9_1(s8[4], s9[5], s9[4], u9_5);
swap sw9_2(s9[3], s9[4], u9_3, out_median);
endmodule

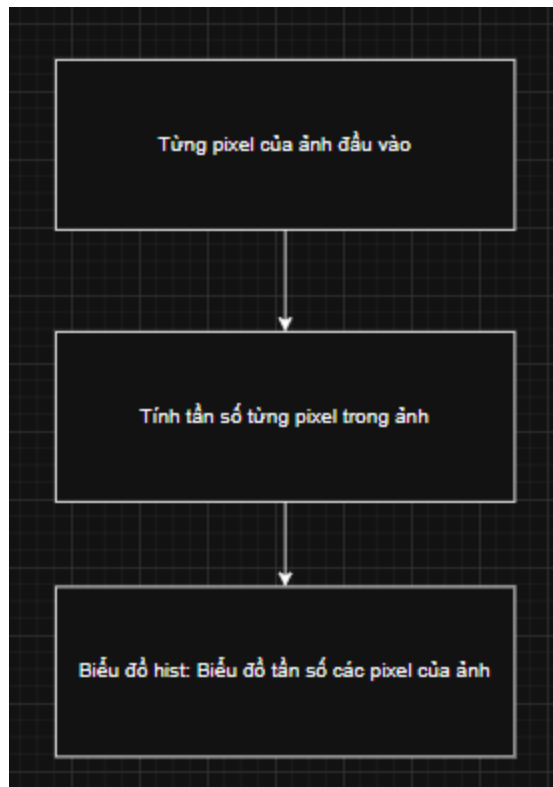
```

- Netlist viewer



3.2.3 Module Histogram Equalization

- Verilog Histogram:



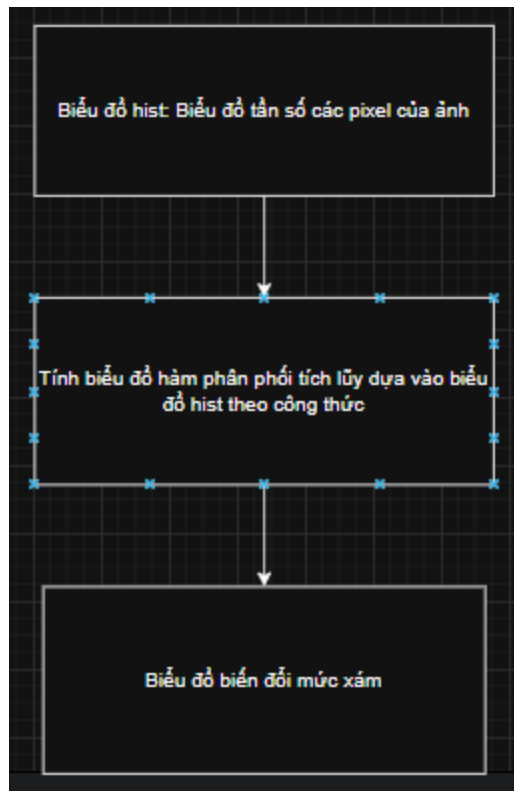
```
module histogram #(
parameter GRAYSCALE = 256,
parameter WIDTH = 698,
parameter HEIGHT = 463,
parameter TOTAL_PIXELS = WIDTH * HEIGHT ) (
input clk,
input rst_n,
input start_hist,
input [7:0] image_input,
output reg hist_done,
output reg [20 * GRAYSCALE -1:0] hist_out
);
reg read;
integer i, x, y;
always @(posedge clk or negedge rst_n) begin
if (!rst_n) read <= 0;
else
if (start_hist && !read)
begin
read <= 1;
end
```

```

end
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for (i = 0; i < 256; i = i + 1)
            hist_out[i*20+: 20] <= 0;
            hist_done <= 0;
            doing <=0;
            x <= 0;
            y <= 0;
        end
    else if (start_hist && !hist_done && read) begin
        hist_out[image_input20+: 20] <= hist_out[image_input20+:20] + 1;
        if (x < WIDTH - 1)
            x <= x + 1;
        else begin
            x <= 0;
            if (y < HEIGHT - 1)
                y <= y + 1;
            else
                begin
                    hist_done <= 1;
                end
            end
        end
    end
end
end
endmodule

```

- Verilog cdf_lut



```
module cdf_lut #(
parameter GRAYSCALE = 256,
parameter WIDTH = 698,
parameter HEIGHT = 463,
parameter TOTAL_PIXELS = WIDTH * HEIGHT
)
(
input clk,
input rst_n,
input cdf_lut_start,
input [20 * GRAYSCALE -1:0] hist_in,
output reg lut_done,
output reg [8 * GRAYSCALE -1:0] lut_out
);
integer i;
integer file_out;

reg [31:0] cdf;
reg write_done;
reg [7:0] count;

always @(posedge clk or negedge rst_n) begin
if (!rst_n) begin
```

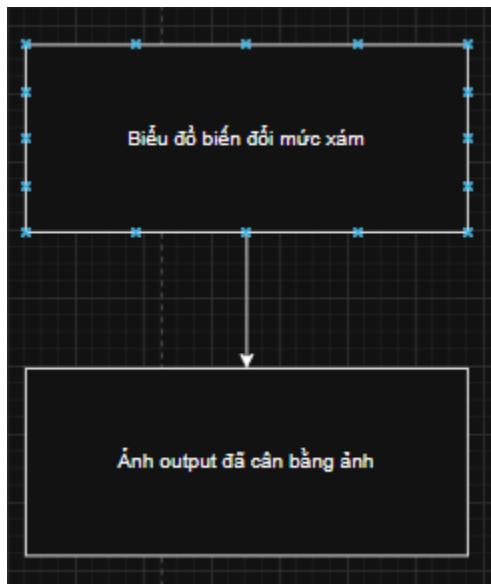


```

lut_done <= 0;
cdf <= 0;
write_done <= 0;
count <= 0;
end
else if (cdf_lut_start && !lut_done) begin
cdf = cdf + hist_in[count * 20 +:20];
lut_out[count * 8 +: 8] = (cdf * 255) / TOTAL_PIXELS;
count = count + 1;
if (count == 256) lut_done = 1;
end
end
endmodule

```

- Verilog equalize



```

module equalize #(
parameter GRAYSCALE = 256,
parameter WIDTH = 698,
parameter HEIGHT = 463,
parameter TOTAL_PIXELS = WIDTH * HEIGHT
)
(
input clk,
input rst_n,
input equalize_start,
input [7:0] image_input,
input [8 * GRAYSCALE -1:0] lut_in,
output reg write_done,

```

```

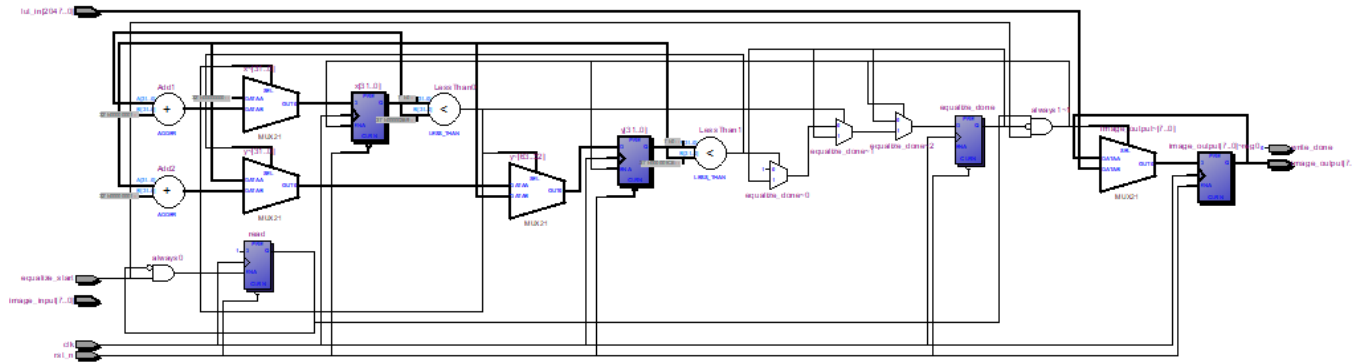
    output reg [7:0] image_output
);

reg [7:0] image_memory [0:TOTAL_PIXELS-1];

reg [7:0] reg_anhoutput [0:TOTAL_PIXELS-1];
integer file_out;
integer i;
integer x, y;
reg read;
reg equalize_done;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) read <= 0;
    else
        if (equalize_start && !read) begin
            read <= 1;
        end
end
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        equalize_done <= 0;
        x <= 0;
        y <= 0;
        write_done <= 0;
    end
    else if (equalize_start && !equalize_done && read) begin
        image_output[7:0] <= lut_in[image_memory[y*WIDTH+x]*8+:8];
        if (x < WIDTH - 1)
            x <= x + 1;
        else begin
            x <= 0;
            if (y < HEIGHT - 1)
                y <= y + 1;
            else
                equalize_done <= 1;
        end
    end
end
end
endmodule

```

- Netlist Viewer



3.3 Bộ nhớ và lưu trữ ảnh

- Trong hệ thống, bộ nhớ SDRAM được sử dụng để lưu toàn bộ dữ liệu ảnh đầu vào, cũng như ảnh sau khi xử lý của từng khối phần cứng. Nó làm vùng đệm trung gian cho quá trình xử lý ảnh, cũng như cho phép truy xuất dữ liệu với dung lượng lớn và tốc độ cao.
- Trong đề tài, SDRAM trên kit DE2 được kết nối với hệ thống Nios II thông qua SDRAM Controller trong Qsys. Dữ liệu ảnh grayscale được lưu dưới dạng 8-bit/pixel, mỗi pixel tương ứng 1 byte trong SDRAM.
- Ví dụ bố trí dữ liệu trong SDRAM

Vùng nhớ	Chức năng
0x00800000	Ảnh đầu vào
0x00800000 + IMAGE_SIZE	Ảnh sau Median Filter
0x00800000 + 2*IMAGE_SIZE	Ảnh sau Histogram Equalization

- Cách ghi dữ liệu từ PC vào SDRAM của DE2
 - + Để truyền dữ liệu ảnh từ máy tính xuống SDRAM, hệ thống sử dụng công cụ System Console của Intel Quartus kết hợp với Tcl Script và IP JTAG Avalon Master Bridge trong Qsys.
 - + Dữ liệu ảnh được chuyển đổi từ ảnh .jpg sang file nhị phân .bin, trong đó mỗi pixel grayscale tương ứng với 1 byte dữ liệu.

- + Sau đó, Tcl Script sử dụng lệnh `master_write_from_file` để ghi trực tiếp dữ liệu từ file .bin xuống SDRAM thông qua giao tiếp JTAG mà không cần truyền qua UART.

- + Script ghi dữ liệu xuống SDRAM

```
set master_paths [get_service_paths master]

set master_path [lindex $master_paths 0]

open_service master $master_path

master_write_from_file $master_path
"C:/Nam_3_HK2_2025_2026/HDL/Do_an_mon/Anhinput.bin" 0x00800000

puts "Da nap xong anh xuong SDRAM!"

close_service master $master_path
```

- Cách đọc dữ liệu từ SDRAM của DE2 sang PC

- + Sau khi các khối xử lý ảnh hoàn tất (ví dụ Median Filter hoặc Histogram Equalization), dữ liệu ảnh kết quả được lưu tại vùng SDRAM output.
- + System Console sử dụng lệnh `master_read_to_file` để đọc dữ liệu từ SDRAM và ghi trực tiếp thành file .bin trên máy tính.

- + Script đọc dữ liệu từ SDRAM

```
set master_paths [get_service_paths master]

set master_path [lindex $master_paths 0]

open_service master $master_path

# Kích thước ảnh

set WIDTH 430
set HEIGHT 554

set total_bytes [expr $WIDTH * $HEIGHT]

# Địa chỉ kết quả median
```

```
set RESULT_ADDR [expr 0x00800000 + $total_bytes]

# Doc SDRAM -> file

master_read_to_file $master_path
"C:/Nam_3_HK2_2025_2026/HDL/Do_an_mon/Anhoutput.bin" $RESULT_ADDR
$total_bytes

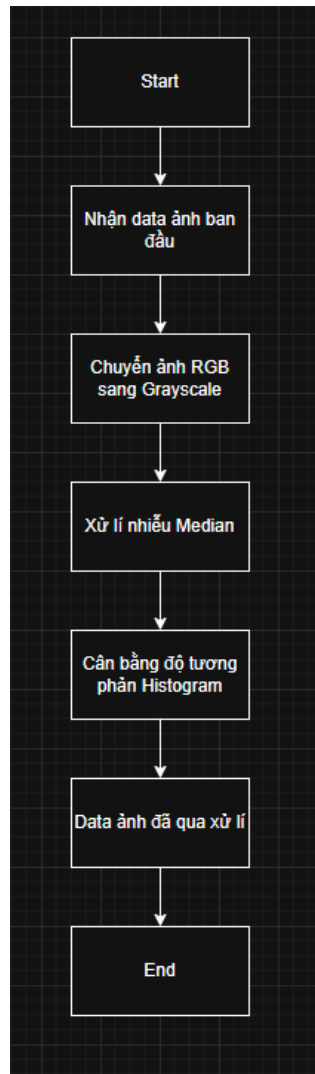
puts "Da lay anh ket qua tu SDRAM ve PC thanh cong!"

close_service master $master_path
```

- Sau khi đọc dữ liệu từ SDRAM, file .bin được chuyển lại thành ảnh bằng Python và thư viện Pillow.

3.4 Lưu đồ thuật toán

Hệ thống xử lý ảnh được thực hiện theo các bước sau:



Mô tả hoạt động:

1. Nhận dữ liệu ảnh từ máy tính.
2. Chuyển đổi ảnh RGB sang grayscale.
3. Thực hiện lọc nhiễu bằng Median Filter.
4. Tăng độ tương phản bằng Histogram Equalization.
5. Lưu ảnh kết quả.

CHƯƠNG 4. KẾT QUẢ MÔ PHỎNG VÀ THỰC NGHIỆM

4.1 Mô phỏng

Công cụ mô phỏng: Phần mềm ModelSim-Altera

Mô phỏng thuật toán gray scale:

-Ảnh ban đầu:

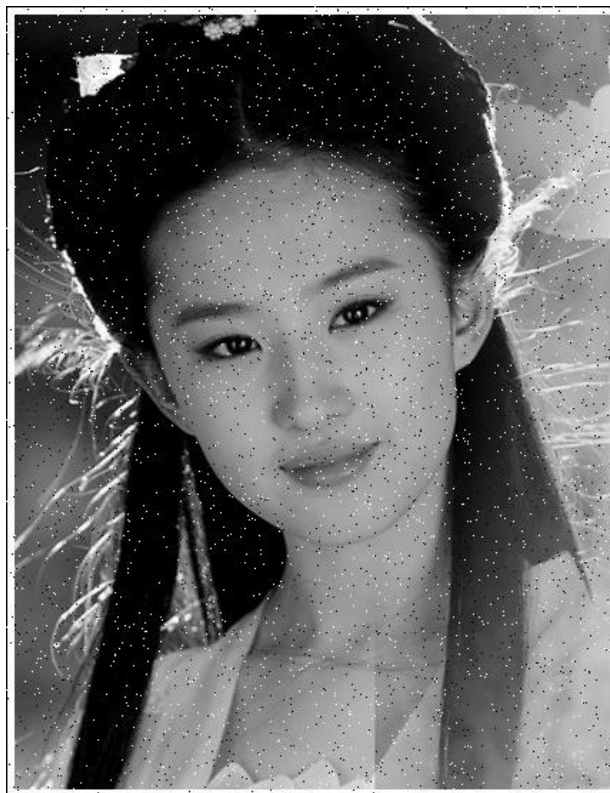


Ảnh sau khi chuyển thành ảnh xám:



Mô phỏng thuật toán median filter:

- Ảnh dùng để mô phỏng:



- Ảnh ban đầu sau khi lọc nhiễu median filter:



Mô phỏng thuật toán toán histogram 1:

- Ảnh ban đầu:



- Ảnh ban đầu sau khi cân bằng histogram:



Mô phỏng thuật toán histogram 2:

- Ảnh ban đầu:

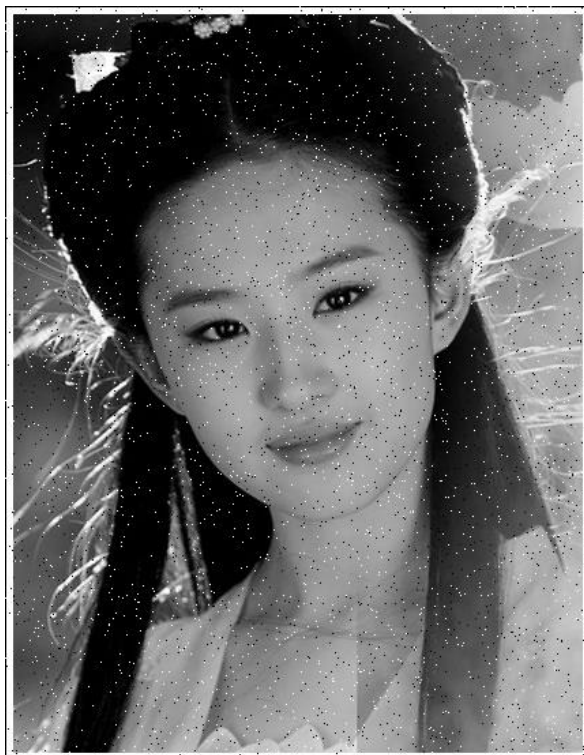


- Ảnh sau khi cân bằng histogram:



Mô phỏng thuật toán histogram và median filter:

-Ảnh ban đầu:



-Ảnh sau khi lọc nhiễu median filter và cân bằng histogram:



4.2 Thực nghiệm

Video nạp kit:

https://drive.google.com/file/d/1IRie2UMnT63qM29LWqt8eX0Edy-_F5cS/view?usp=drive_link

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Trong đề tài này, hệ thống xử lý ảnh cơ bản trên FPGA đã được nghiên cứu, thiết kế và thực hiện thành công trên kit FPGA DE2. Các thuật toán xử lý ảnh gồm chuyển đổi ảnh grayscale, lọc nhiễu bằng Median Filter và tăng cường độ tương phản bằng Histogram Equalization đã được xây dựng bằng ngôn ngữ Verilog HDL.

Đề tài đã thực hiện được các nội dung sau:

- Nghiên cứu cơ sở lý thuyết về xử lý ảnh số và kiến trúc FPGA.
- Thiết kế các module xử lý ảnh bằng Verilog HDL.
- Mô phỏng và kiểm tra chức năng hoạt động của từng module.
- Truyền dữ liệu ảnh từ máy tính xuống FPGA thông qua JTAG UART.
- Lưu trữ và xử lý dữ liệu ảnh trên SDRAM.
- Kiểm thử và đánh giá kết quả xử lý ảnh sau khi thực hiện các thuật toán.

Kết quả đạt được cho thấy:

- Ảnh grayscale được chuyển đổi chính xác từ ảnh RGB.
- Median Filter giúp giảm nhiễu muối tiêu hiệu quả.
- Histogram Equalization cải thiện độ tương phản của ảnh.
- Hệ thống hoạt động ổn định trên kit FPGA DE2.
- Các thuật toán được thực hiện bằng phần cứng giúp tăng tốc độ xử lý so với thực hiện bằng phần mềm thông thường.

Thông qua đề tài, sinh viên đã nâng cao kiến thức về:

- Thiết kế phần cứng số bằng Verilog HDL.
- Hệ thống FPGA và bộ nhớ SDRAM.
- Kiến trúc xử lý ảnh trên phần cứng.
- Quy trình mô phỏng, tổng hợp và triển khai hệ thống trên FPGA.

5.2 Hướng phát triển

Mặc dù đề tài đã đạt được các mục tiêu cơ bản, hệ thống vẫn còn nhiều hướng phát triển trong tương lai như:

- Xây dựng hệ thống xử lý video realtime thay cho ảnh tĩnh.
- Tích hợp hiển thị trực tiếp qua VGA hoặc HDMI.

- Nâng cấp thuật toán Histogram Equalization thành CLAHE để cải thiện chất lượng ảnh tốt hơn.
- Thực hiện các thuật toán phát hiện biên như Sobel, Prewitt hoặc Canny.
- Tối ưu tốc độ và tài nguyên phần cứng của hệ thống.
- Thiết kế kiến trúc pipeline để tăng hiệu năng xử lý.
- Kết hợp các mô hình AI hoặc CNN accelerator trên FPGA cho các ứng dụng nhận dạng ảnh.
- Mở rộng hệ thống sang các dòng FPGA có tài nguyên lớn hơn.

TÀI LIỆU THAM KHẢO

[1] Rafael C. Gonzalez, Richard E. Woods, *Digital Image Processing*, Pearson Education.

[2] Pong P. Chu, *FPGA Prototyping by Verilog Examples*, Wiley.

[3] Altera Corporation, *DE2 Development and Education Board User Manual*.

[4] Samir Palnitkar, *Verilog HDL – A Guide to Digital Design and Synthesis*.